# The Devil Is in the Interface: Evaluating How Tool Architecture Shapes Coding Agent Behavior

**Xiangzhe Xu**[1*] **Hamidreza Saghir**[2] **Qianhui Wu**[2] **Marc-Alexandre Côté**[2]

**Tong Wang**[2] **Kiran Lakkaraju**[2] **Kexin Pei**[3] **Xiangyu Zhang**[1]

[1]Purdue University [2]Microsoft Research [3]The University of Chicago
{xzx, xyzhang}@purdue.edu kpei@uchicago.edu
{hsaghir, qianhuiwu, macote, tong.wang, kiran.lakkaraju}@microsoft.com
[*] Contribution during an internship at Microsoft Research.

arXiv:2608.11386v1 [cs.SE] 11 Aug 2026

## Abstract

As large language models continue to improve, agentic systems are becoming increasingly important, and tools are a key design dimension because they determine how agents access information and take action in their environments. Prior work on agent tooling has primarily focused on expanding what agents can do, but has paid less systematic attention to how those capabilities are organized and exposed to the model. We refer to this latter design dimension as *tool architecture*. We study tool architecture in coding agents through controlled experiments on repository-level issue fixing, comparing six tool architectures that hold the underlying information and actions similar while varying how they are organized and exposed to the model, across three actors and a total of 11,700 trajectories. Our experiments show that, even when tools provide similar capabilities, tool architecture changes agent behavior: Compared to a basic architecture where the agent has only the bash tool, more structured low-level interfaces improve consistency across repeated attempts by up to 4.7×; natural-language search broadens repository exploration and increases access to relevant files by more than 11%; and Python CodeAct-style interfaces achieve similar task performance with 41.6% fewer steps and 56.3% lower token usage. By contrast, lightweight text-based cognitive-scaffolding tools, such as tools that let the agent record intermediate reasoning, have limited effect on actor behavior.

## 1 Introduction

As large language models continue to improve, agentic systems are becoming increasingly important (Microsoft, 2025; BCG, 2025; McKinsey, 2025b;a). Beyond continued model scaling, growing attention has turned to harness engineering (Anthropic, 2026a; OpenAI, 2026; Anthropic, 2026b), which studies how the surrounding system enables a model to interact with external environments and complete complex tasks. Within this broader space, tool design is a particularly important dimension (Wang et al., 2024a; Yang et al., 2024; Yao et al., 2024; Wang et al., 2025b) because tools define how an agent accesses information and interacts with the environment.

Most prior work on agent tools has focused on expanding what the agent can do by giving it new capabilities. In coding agents, this often means adding components such as program analysis modules (Zhang et al., 2024; Ruan et al., 2025), mechanisms for rolling out and comparing multiple candidate solutions (Ma et al., 2024; Gao et al., 2025), or precomputed semantic embeddings that help the agent retrieve relevant code (Ma et al., 2024; Ouyang et al., 2024; Liu et al., 2025b). These additions improve performance by giving the agent access to information or actions that would otherwise be unavailable or difficult to obtain.

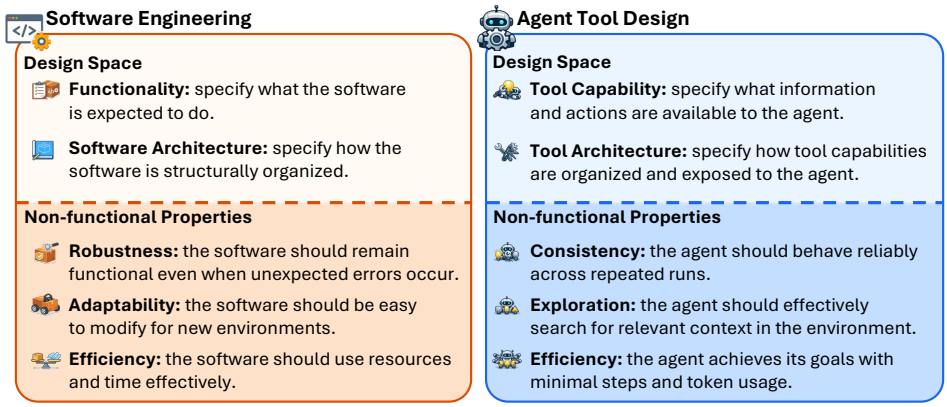



Figure 1: Analogy between software engineering (SE) and agent tool design. In SE, software architecture is complementary to functionality and shapes non-functional properties such as robustness (Randell, 1975; Parnas, 1972; Williams & Smith, 1998). Our study focuses on the analogous question in agent tool design. We find that, for tools with similar capabilities, different tool architectures, that is, different ways of organizing and exposing information and actions to the model, can affect consistency, exploration, and efficiency.

However, tool capability is not the whole story (Gloaguen et al., 2026; Garg & Huang, 2026). Even when two agents can access similar information and take similar actions, the way those capabilities are organized and exposed to the model can still meaningfully affect behavior (Yang et al., 2024). It can influence how consistently the agent solves the same task across runs (Rabanser et al., 2026), how broadly or narrowly it explores a repository (Gloaguen et al., 2026; Li et al., 2026), and how efficiently it uses steps and tokens (Lulla et al., 2026). We refer to this design dimension as *tool architecture*.

Figure 1 illustrates our conceptual analogy to software engineering (Parnas, 1972). Tool capability is analogous to software functionality: it describes what information and actions are available to the agent. Tool architecture, by contrast, is analogous to software architecture or interface design (Perry & Wolf, 1992; Kazman & Bass, 1994): it describes how those capabilities are organized and exposed to the model. Our experiments show that this distinction matters in practice for agent tools. Even when two tool setups support similar tasks, they can still lead the model to behave differently because they present similar underlying capabilities through different interfaces.

In practice, however, capability and architecture are often entangled (Garg & Huang, 2026; Ouyang et al., 2024). For example, a tool for semantic retrieval over a code repository both introduces additional capability through embedding-based retrieval and changes the search architecture by replacing lower-level grep-style interaction with a more abstract natural-language interface. As a result, if such a tool improves performance, it is unclear where the gain should be attributed. Our experiments suggest that part of the gain can arise from architecture alone: even without embeddings, implementing semantic search through bash operations but exposing it through a more natural-language-oriented retrieval interface leads agents to explore more than 11% more files in the repository.

To isolate the effect of tool architecture, we design six tool architectures based on commonly adopted tool designs in existing coding agents (OpenHands, 2026; Anthropic, 2026c; Gao et al., 2025; Antoniades et al., 2024; Smolagents, 2026; Yang et al., 2024; Code, 2026a) and implement them so that they expose similar information and similar capability to interact with the environment. We study two dimensions of architecture. The first is the *level of abstraction*: some interfaces expose low-level operations (e.g., file editing) directly, some provide natural-language search interfaces, and some let the agent write executable code instead of issuing individual tool calls. The second is *cognitive scaffolding*: those tools provide interfaces for recording intermediate reasoning or task-specific assumptions. Here are our key contributions and findings:

- We design controlled experiments for tool architecture by implementing six representative architectures with similar capabilities.
- We evaluate three actor models over 11,700 coding-agent trajectories in total. The experiments find that when tool capabilities are kept similar, overall task performance remains broadly similar across architectures.
- More structured low-level interfaces improve repeated-attempt consistency, by up to 4.7×. Natural-language search interfaces broaden repository exploration and increase access to plausibly relevant files by more than 11%, but also introduce additional noise. Code-execution-based interfaces achieve similar task performance with 41.6% fewer steps and 56.3% lower token cost.
- By contrast, lightweight text-based cognitive-scaffolding tools have limited effect because they rarely change the actors' underlying reasoning behavior.
- We release our code and data at `https://github.com/XZ-X/tool-arch-study.git`.

## 2 Setups

### 2.1 Tool Setups

We study six tool architectures that are commonly adopted in existing coding agents; Table 1 summarizes the six setups and representative agents with corresponding design choices. Our goal is to study tool architecture, so we design these setups to keep their capabilities as similar as possible. We use `BashOnly` as the baseline, where the agent interacts with the repository only through a general-purpose shell. From this baseline, we vary tool architecture along two dimensions.

The first dimension is the *level of abstraction*: `Atomic` exposes common low-level operations as explicit tools, `NLSearch` provides a natural-language search interface for repository exploration, and `Python` lets the agent write executable Python code instead of issuing individual tool calls.

The second is *cognitive scaffolding*: `HypoTrack` lets the agent record and update hypotheses about the task, while `Scratchpad` lets the agent write down intermediate reasoning during problem solving. Note that we implement these scaffolds as lightweight text-based tools so that they provide an explicit channel for actor-written intermediate state without adding retrieval (Ouyang et al., 2024), memory management (Wang et al., 2026a), new task information (Ruan et al., 2025), or enforcing a different reasoning policy (Antoniades et al., 2024). This design lets us isolate whether simple cognitive-scaffolding interfaces change actor behavior under otherwise similar capabilities.

Figure 2 shows concrete examples of the interaction style in each setup.

**Baseline: BashOnly.** The baseline setup exposes only a general-purpose bash interface. The agent interacts with the repository through unconstrained shell commands for navigation, search, file inspection, editing, and execution. This setup offers broad flexibility but little structure: the model must compose low-level commands to carry out even routine subtasks. We use it as the reference point because it provides a minimally structured yet widely used (Xia et al., 2025; Jimenez et al., 2023; Yang et al., 2024) coding-agent environment.

**Level of abstraction: Atomic.** The *Atomic* setup augments bash with a small collection of simple tools for common localized operations, such as repository search, bounded file viewing, targeted string replacement, and file creation. We include this setup because prior work (Yang et al., 2024) suggests that clear, simple tool interfaces can improve agent performance, and such structured high-frequency primitives are common (OpenHands, 2026; Anthropic, 2026c; Yang et al., 2024; Gao et al., 2025) in mainstream coding agents. The *Atomic* setup does not extend bash capability; it only repackages common shell actions into simple tools (see Table 3 in the Appendix for the mapping between atomic tools we adopted and their bash equivalents).

**Level of abstraction: NLSearch.** The *NLSearch* setup augments bash with a tool that takes a natural-language query as input and returns relevant code snippets. We include this

Table 1: A taxonomy of the tool architectures studied in this work and representative coding agents with corresponding tool designs.

| Category | Tool Arch. | Representative Agents |
|---|---|---|
| Baseline | BashOnly | SWE-Bench bash only leaderboard (Jimenez et al., 2023), Live-SWE-agent (Xia et al., 2025), Mini-SWE-Agent (Yang et al., 2024). |
| Level of Abstraction | Atomic | OpenHands (OpenHands, 2026), Claude Code (Anthropic, 2026c), TRAE (Gao et al., 2025), SWE-Agent (Yang et al., 2024). |
| | NLSearch | Augment Code context engine (Code, 2026b). |
| | Python | Smolagents (Smolagents, 2026). |
| Cognitive Scaffolding | HypoTrack | Sequential thinking tool (Gao et al., 2025; Anthropic, 2026c; OpenHands, 2026; Code, 2026a), SWE-Search (Antoniades et al., 2024). |
| | Scratchpad | Sequential thinking tool (Gao et al., 2025; Anthropic, 2026c; OpenHands, 2026; Code, 2026a). |

setup because it captures the interface of commonly used semantic search capabilities (Code, 2026b; Liu et al., 2025b; Ma et al., 2024) in coding agents, which let the model express retrieval needs at a higher level than raw shell commands. To avoid exceeding the capability of bash, however, our implementation does ***not*** use semantic indexing or embedding-based retrieval. Instead, we implement *NLSearch* as a subagent with the same actor model, which takes a natural-language query and uses bash commands such as `grep` to iteratively search for and return potentially relevant snippets.

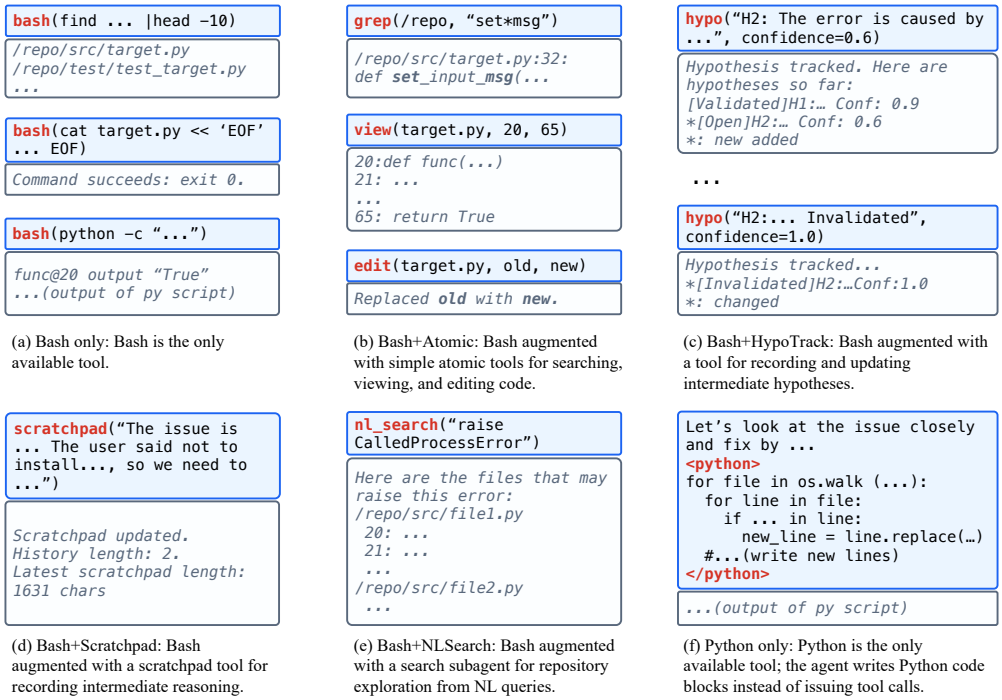
**Level of abstraction: Python.** The *Python* setup replaces explicit tool calls with executable Python blocks: instead of invoking bash or specialized tools, the agent writes Python code to traverse the repository, inspect files, edit content, and run checks. We include this setup because code execution has become a common agent interface, especially in CodeAct-style systems (Wang et al., 2024b; OpenHands, 2026; Smolagents, 2026). This setup also does not exceed the capability of bash in practice, since agents with shell access can already write and execute Python scripts to carry out the same operations; the difference is therefore one of interaction style rather than underlying capability. Appendix B.5 further validates this design choice: in a manual inspection of 100 sampled Python actions, 97% correspond to operations already available in `BashOnly`, either as direct bash-side operations or as Python scripts that `BashOnly` actors commonly create and run from bash.

**Cognitive scaffolding: HypoTrack.** The *HypoTrack* setup adds a simple hypothesis-tracking tool to the bash environment that lets the agent explicitly record a hypothesis and its confidence, and later update the hypothesis status. We include this tool because mainstream agents use similar `thinking` tools to track branches (Gao et al., 2025; Anthropic, 2026c), and because coding agents often benefit from exploring multiple paths (Gao et al., 2025), though such behavior is usually induced by broader system design (Antoniades et al., 2024) rather than a single tool call. We want to isolate whether that lightweight scaffolding alone can encourage broader exploration. To keep *HypoTrack* aligned with the capability of bash, the tool only records a hypothesis and its status, both of which could also be expressed directly by a bash-only agent in its reasoning text.

**Cognitive scaffolding: Scratchpad.** The *Scratchpad* setup adds a simple tool that lets the agent output free-form thinking content. We include it because similar scratchpad-style interfaces are common in mainstream agents (Gao et al., 2025; Anthropic, 2026c; OpenHands, 2026; Code, 2026a). To keep *Scratchpad* aligned with the capability of bash, the tool only accepts free-form text and returns a confirmation message, adding no capability beyond what could already be written in the agent's reasoning text.

### 2.2 Evaluation Protocol

We evaluate agents on issue-resolution tasks from SWE-bench Live (Zhang et al., 2025), a contamination-controlled benchmark for repository-level bug fixing. Each instance provides an issue description and a code repository snapshot, and the agent must generate a patch




(a) Bash only: Bash is the only available tool.

(b) Bash+Atomic: Bash augmented with simple atomic tools for searching, viewing, and editing code.

(c) Bash+HypoTrack: Bash augmented with a tool for recording and updating intermediate hypotheses.

(d) Bash+Scratchpad: Bash augmented with a scratchpad tool for recording intermediate reasoning.

(e) Bash+NLSearch: Bash augmented with a search subagent for repository exploration from NL queries.

(f) Python only: Python is the only available tool; the agent writes Python code blocks instead of issuing tool calls.

Figure 2: Examples of tool architectures evaluated in this work. The panels illustrate how similar capabilities are exposed to the agent through different interfaces in each setup.

that resolves the issue. Outputs are evaluated against hidden human-authored test cases, which are not exposed to the agent.

To make repeated evaluation tractable under our budget, we construct a subset of SWE-bench Live by randomly sampling 25 of its 100 repositories and retaining at most five issues per repository, yielding 65 problem instances in total. For every actor–setup pair, we execute 10 independent rollouts per instance (i.e., attempt at solving the task). Using repeated rollouts lets us measure not only aggregate task performance, but also behavioral properties such as consistency across reruns and diversity in exploration and final solutions.

We evaluate three actor models: Qwen3Coder-30B (Yang et al., 2025), Kimi K2.5 (Team et al., 2026), and Claude Sonnet 4.5 (Anthropic, 2026d). We choose them to cover a diverse set of coding agents, including open-weight and proprietary access, different model scales, and coding- and agent-oriented training.

We also evaluate whether the main tool-architecture effects extend beyond this SWE-bench Live subset. Specifically, we consider additional coding tasks covering issue resolving, feature implementation, and debugging. We use SWE-bench Verified (Jimenez et al., 2023) for issue resolving and SWE-bench Pro (Deng et al., 2025) for feature implementation. For debugging, we construct a separate SWE-bench split by selecting instances whose issue descriptions contain full stack traces. Due to budget constraints, we run this additional study with the two open-weight actors, `Qwen3Coder-30B` and `Kimi-K2.5`, and focus on the pairwise comparisons most directly tied to our main findings. For each actor–task–setup combination, we run 5 independent rollouts per instance. Appendix B.6 reports the detailed results.

### 2.3 Evaluation Dimensions and Metrics

We evaluate tool architectures along four complementary dimensions: *task resolve rate*, *consistency*, *exploration*, and *efficiency*. Below, we describe the motivation and intuition of each metric, and discuss the formal definitions in Appendix A.

**Task resolve rate.** We first report the task resolve rate as the basic measure of whether an agent solves a task. It captures the fraction of repeated attempts that successfully resolve the benchmark problem instance, averaged across instances. This metric tells us how effective a setup is overall.

**Consistency.** A reliable coding agent should not only solve a task, but should achieve success consistently across repeated attempts. Consistency matters because users care about whether the same setup can be trusted to reproduce success, rather than exhibiting success only intermittently (Gupta, 2026; Rabanser et al., 2026). This is especially important in longer-horizon coding tasks, where even a small mistake can cascade into later decisions. We therefore measure consistency by asking how often success remains stable across multiple repeated attempts of the same problem instance. The metric is typically denoted as `pass^k` (Yao et al., 2024). In our experiments, we report this metric for repeated groups of size $k \in \{5, 7, 9\}$.

**Exploration.** Recent work suggests that coding agents often struggle to acquire repository-level context effectively and may under-explore the codebase on repository-grounded tasks (Li et al., 2026). Motivated by this observation, we study whether different tool architectures induce different exploration behavior. In particular, we ask whether some tool architectures encourage the agent to explore more diverse parts of the repository across repeated attempts and to produce different candidate solutions. We measure exploration from two complementary perspectives: how much repeated attempts differ in the files they read, and how much their final patches differ from one another. Concretely, we use Jaccard (Jaccard, 1901) distance to measure diversity in file access and CodeBLEU (Ren et al., 2020) distance to measure diversity in generated solutions; see Appendix A for details.

**Efficiency.** Besides consistency and exploration, the cost of a coding agent is also important in real deployment (Bui, 2026; Xiao et al., 2026; Pan et al., 2025; Lulla et al., 2026). We measure efficiency using three trajectory-level statistics: input tokens, output tokens, and trajectory length (i.e., the number of interaction steps). We report the average of each quantity for every actor–setup pair and interpret these measurements jointly with task performance, since lower token usage and fewer steps are not necessarily preferable if they come at the cost of substantially worse task resolution.

## 3 Evaluation Results

### 3.1 Task Resolve Rate

The overall task resolve rate is similar across tool architectures for a given actor model. This is expected, since we intentionally design the tool architectures to minimize capability differences relative to the BashOnly setup, allowing us to isolate the effects of architecture on non-functional properties. See Appendix B.1 for details.

### 3.2 Consistency

Table 2 shows a clear consistency result: `Atomic` is the only setup that improves `pass^k` relative to `BashOnly` for all three actors. Because our goal is to isolate the effect of tool architecture within each actor, the key comparisons are the changes from `BashOnly` for the same actor, rather than the absolute `pass^k` values alone. We can see that `Atomic` improves `pass^5`, `pass^7`, and `pass^9` for `Qwen3Coder-30B`, `Kimi-K2.5`, and `Sonnet-4.5`, making it the only setup with a uniformly positive consistency effect.

The magnitude of this gain is also actor-dependent. The improvement is largest for the weakest actor, `Qwen3Coder-30B`, where `Atomic` increases `pass^k` by 0.059–0.074 depending on $k$. The gains are smaller but still positive for the stronger actors: 0.013–0.014 for `Kimi-K2.5` and 0.017–0.031 for `Sonnet-4.5`. This pattern suggests that packaging recurring low-level repository actions into clearer, more structured primitives is especially helpful for weaker actors, whose trajectories appear more vulnerable to small execution mistakes that can accumulate across an attempt. For the two open-weight actors, we additionally evaluate `BashOnly` and `Atomic` in Appendix B.2 using extended repeated evaluation with three times more attempts, and the results show that these gains remain stable.

Table 2: Consistency results across actor models. Each cell reports absolute pass^k; parenthesized values denote the change relative to BashOnly. Background color indicates change relative to BashOnly: blue = improvement, orange = decline, gray = baseline.

| Actor | Setup | pass^5 | pass^7 | pass^9 |
|---|---|---|---|---|
| Qwen3Coder-30B | BashOnly | 0.046 | 0.031 | 0.020 |
| | Atomic | 0.106 (**+0.059**) | 0.097 (**+0.067**) | 0.094 (**+0.074**) |
| | NLSearch | 0.051 (+0.005) | 0.039 (+0.008) | 0.032 (+0.012) |
| | HypoTrack | 0.040 (-0.006) | 0.032 (+0.002) | 0.031 (+0.011) |
| | Scratchpad | 0.029 (-0.017) | 0.021 (-0.010) | 0.017 (-0.003) |
| | Python | 0.016 (-0.030) | 0.006 (-0.025) | 0.002 (-0.018) |
| Kimi-K2.5 | BashOnly | 0.290 | 0.277 | 0.266 |
| | Atomic | 0.304 (**+0.014**) | 0.289 (**+0.013**) | 0.280 (**+0.014**) |
| | NLSearch | 0.258 (-0.031) | 0.246 (-0.031) | 0.235 (-0.031) |
| | HypoTrack | 0.282 (-0.008) | 0.272 (-0.005) | 0.265 (-0.002) |
| | Scratchpad | 0.268 (-0.022) | 0.257 (-0.020) | 0.249 (-0.017) |
| | Python | 0.265 (-0.024) | 0.256 (-0.021) | 0.249 (-0.017) |
| Sonnet-4.5 | BashOnly | 0.296 | 0.270 | 0.252 |
| | Atomic | 0.313 (+0.017) | 0.297 (+0.027) | 0.283 (+0.031) |
| | NLSearch | 0.314 (+0.018) | 0.303 (**+0.033**) | 0.295 (**+0.043**) |
| | HypoTrack | 0.315 (**+0.020**) | 0.291 (+0.021) | 0.271 (+0.018) |
| | Scratchpad | 0.279 (-0.017) | 0.260 (-0.010) | 0.249 (-0.003) |
| | Python | 0.283 (-0.013) | 0.266 (-0.004) | 0.252 (0.000) |

The remaining setups do not show the same cross-actor regularity. NLSearch and HypoTrack improve consistency for some actors but reduce it for others, while Scratchpad and Python are generally neutral to negative. By comparison, Atomic is the only setup with uniformly positive pass^k deltas across all three actors, and its gains are largest for Qwen3Coder-30B.

**Mechanism: Atomic improves consistency by reducing interaction errors.** A plausible mechanism for Atomic's consistency gains is that it reduces low-level environment-interaction errors. This interpretation is motivated by prior work in system reliability and software dependability, which often analyzes failures through chains linking faults, errors, and observed failures (Avizienis et al., 2004; Gupta, 2026; Rabanser et al., 2026). In our experiments, Atomic is associated with fewer such errors than BashOnly, especially for the weakest actor, and most of the reduction comes from error types that a more structured interface should directly help avoid, such as malformed commands and broken edits. This pattern suggests that packaging recurring low-level actions into explicit primitives can improve stability across repeated attempts by reducing execution-level mistakes. Appendix B.3 provides the full error analysis and category breakdown.

**Finding 1: Atomic improves consistency across actors.**

Compared with BashOnly, Atomic improves consistency for all three actors, with the largest gain for Qwen3Coder-30B. A plausible mechanism is that Atomic reduces low-level environment-interaction errors, which are more common in weaker actors and therefore leave more room for improvement.

### 3.3 Exploration

Exploration matters because coding agents may under-explore repository-level context (Li et al., 2026), and tool architectures that broaden exploration may expose them to different code context and candidate solutions. Figure 3 shows that NLSearch is the only setup that consistently increases *read diversity* across all three actors. The gains are substantial for all models, whereas other setups have smaller or more actor-specific effects. By contrast, changes in *solution diversity* are much smaller and less uniform. Across most actor–setup

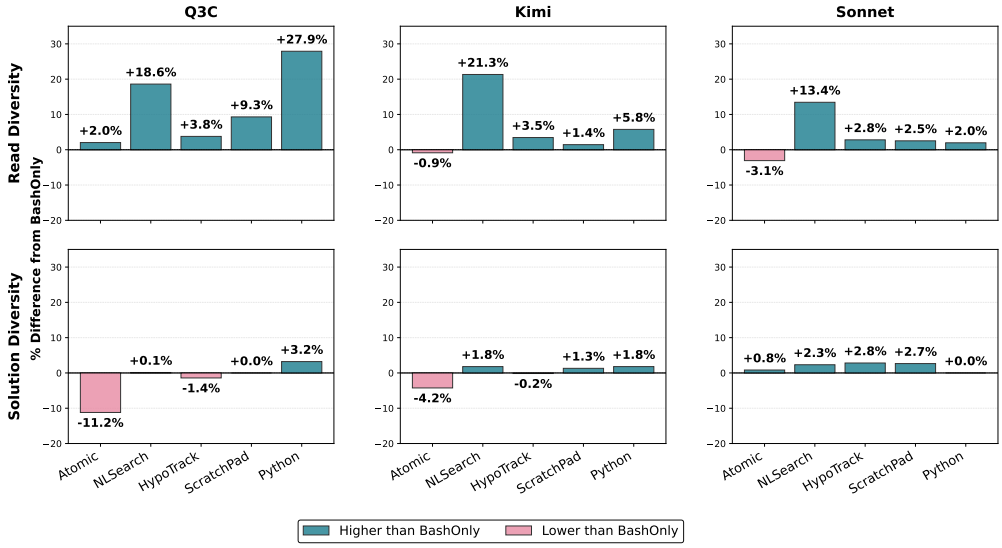



Figure 3: Effect of tool architecture on exploration and final-solution diversity. Bars show the difference from `BashOnly` for each actor–setup pair. The top row reports *read diversity*, measured by pairwise Jaccard distance over files read across repeated attempts of the same problem instance. The bottom row reports *solution diversity*, measured by pairwise CodeBLEU distance between generated patches. `NLSearch` is the only setup that consistently increases read diversity across all three actors, whereas effects on solution diversity are smaller and less uniform.

pairs, repeated attempts under different tool architectures still converge to similar final patches even when they inspect different parts of the repository. Overall, tool architecture has a clearer effect on repository traversal than on the diversity of final solutions.

To better understand this effect, we ask two follow-up questions: (1) what drives the broader exploration under `NLSearch`, and (2) whether that additional exploration is meaningful for the given problem instance. Appendix B.4 suggests that the broader exploration is associated with more diverse early search queries under the natural-language interface than under raw `grep`/`find`-style search. It also shows that the additional exploration improves recall of high-relevance files across all three actors, though with lower precision than `BashOnly`.

**Finding 2: NLSearch improves exploration but not solution diversity.**

NLSearch consistently broadens repository coverage and increases recall of relevant files, but these gains only weakly translate to more diverse final solution strategies. The effect appears to come from more diverse early search queries, which broaden the code context seen during execution.

### 3.4 Efficiency

Besides consistency and exploration, the cost of a coding agent is also important in deployment. Figure 4 shows a clear overall pattern: `Python` is the most efficient setup across all three actors. Relative to `BashOnly`, it achieves similar task performance with substantially lower input-token cost and fewer interaction steps, making it the clearest efficiency win in our study. By contrast, the effect of `Atomic` is actor-dependent. `Atomic` improves efficiency for `Qwen3Coder-30B`, but is less efficient for the stronger actors, `Kimi-K2.5` and `Sonnet-4.5`, where the increase in input tokens is especially pronounced.

**Appendix B.5 suggests that these efficiency differences mainly depend on how well each interface supports compound interaction when tool capabilities are otherwise similar.** The `Python` interface encourages the actor to bundle multiple operations into a single executable

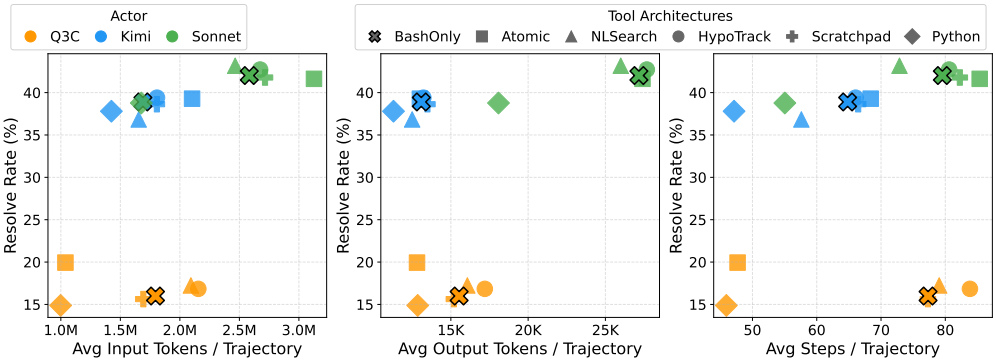



Figure 4: Efficiency comparison across tool architectures. Each point represents one actor–setup pair; points closer to the upper-left of a panel indicate a more favorable efficiency–performance trade-off, achieving higher resolve rate with lower cost. The three panels show resolve rate versus average input tokens, output tokens, and interaction steps per trajectory. `Python` consistently achieves competitive performance with fewer steps and lower token cost, whereas the efficiency effect of `Atomic` depends on the actor: it reduces cost for `Qwen3Coder-30B` but increases cost for `Kimi-K2.5` and `Sonnet-4.5`.

block, which helps reduce both trajectory length and cumulative input-token cost. By contrast, the efficiency effect of `Atomic` depends on the actor. For stronger actors, it can fragment work that would otherwise be combined in bash into more turns, increasing overhead. For `Qwen3Coder-30B`, this fragmentation is smaller, and `Atomic` also reduces low-level interaction errors, which further improves efficiency. Appendix B.5 provides the detailed analysis.

**Finding 3: Tool architectures supporting compound interaction are more efficient.**

`Python` consistently improves efficiency by allowing actors to express more work in a single interaction. `Atomic`, by contrast, is beneficial only when the actor already operates close to an atomic style; otherwise, it fragments compound bash workflows into more turns and raises cumulative token cost.

### 3.5 Why Cognitive Scaffolding Has Limited Effects

The two cognitive-scaffolding setups, `Scratchpad` and `HypoTrack`, have only limited effects in our main evaluation. We interpret this result as specific to the lightweight scaffolds studied here, rather than as a claim about cognitive scaffolding in general. Appendix C suggests that, in our setting, actors usually do not use these tools in ways that substantially change their reasoning. Under `Scratchpad`, actors often produce content that closely matches reasoning already present in `BashOnly`. Under `HypoTrack`, actors rarely maintain multiple competing hypotheses.

**Finding 4: Lightweight cognitive scaffolding has limited effect on actor reasoning.**

Although `Scratchpad` and `HypoTrack` provide explicit interfaces for recording intermediate reasoning, they do not add retrieval, memory management, new task information, or enforce a different reasoning policy. Actors therefore mostly project their original reasoning patterns into these lightweight scaffolds, suggesting that such scaffolds alone have limited effect in our setting.

### 3.6 Generalization to Additional Coding Tasks

We further validate whether the main tool-architecture effects extend beyond our SWE-bench Live subset. Appendix B.6 reports results on additional coding tasks, including issue

resolving, feature implementation, and debugging. The results are directionally consistent with the main findings: `Atomic` improves `pass^k`, `NLSearch` increases read diversity, and `Python` generally reduces input-token cost and trajectory length.

## 4 Related Work

**Empirical studies of coding agents.** Recent empirical work studies coding agents from several perspectives, including task-specific analysis, real-world developer use, planning behavior, and agent context or configuration (Ceka et al., 2026; Majgaonkar et al., 2025; Liang et al., 2025; Horikawa et al., 2025; Watanabe et al., 2026; Wang et al., 2025a; Liu et al., 2025a; Galster et al., 2026; Chatlatanagulchai et al., 2025). These studies provide insights about how coding agents are used in practice, but they offer limited guidance on how the organization and exposure of tools shape coding-agent behavior.

**Empirical studies of tool design.** Another related line of work studies tool use more directly, especially whether agents can retrieve and invoke the correct tool in large and realistic tool ecosystems (Wang et al., 2025b; Fan et al., 2025). However, it primarily evaluates whether an agent can use the right tool rather than how tool architectures affect agent behavior.

**Agent non-functional properties.** Recent work has also emphasized non-functional properties of agents, including efficiency, robustness, and reliability (Gao & Peng, 2025; Wang et al., 2026b; Rabanser et al., 2026). These studies motivate going beyond aggregate task success, but they do not study which parts of the design space produce these properties.

**Tool design in coding agents.** Many coding-agent systems propose concrete tool designs (Pan et al., 2026; Yang et al., 2024; Liu et al., 2025b; Ouyang et al., 2024; Anthropic, 2026c; Antoniades et al., 2024; OpenHands, 2026). These systems directly inspire the tool architectures we study. However, they typically introduce new tools as part of end-to-end agent designs, changing capability and interface at the same time. By contrast, our work isolates tool architecture itself and studies its effect under roughly matched capabilities.

Please refer to Appendix E for detailed discussion of related work.

## 5 Conclusion

Our work isolates the effect of tool architecture in coding agents under similar capabilities. We find that architectures with different levels of abstraction can substantially affect consistency, exploration, and efficiency, whereas cognitive-scaffolding architectures have limited effect in our setting. These results provide practical guidance for coding-agent tool design.

## Acknowledgments

We thank the anonymous reviewers for their valuable comments and suggestions. This research was supported, in part by the Microsoft Internship Program, National Science Foundation (NSF) Awards SHF-1901242, SHF-1910300, Proto-OKN 2333736, IIS-2416835, DARPA VSPELLS - HR001120S0058, and ONR N00014-23-1-2081. Any opinions, findings, and conclusions in this paper are those of the authors only and do not necessarily reflect the views of our sponsors.

## LLM Usage Disclosure

We used large language models to help polish the writing of the paper. The authors reviewed and verified all content.

## References

Anthropic. Demystifying evals for ai agents, 2026a. URL https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents.

Anthropic. Harness design for long-running application development, 2026b. URL https://www.anthropic.com/engineering/harness-design-long-running-apps.

Anthropic. Claude code overview, 2026c. URL https://code.claude.com/docs/en/overview.

Anthropic. Claude sonnet, 2026d. URL https://www.anthropic.com/claude/sonnet.

Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement. *arXiv preprint arXiv:2410.20285*, 2024.

Algirdas Avizienis, J-C Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE transactions on dependable and secure computing*, 1(1):11–33, 2004.

BCG. Momentum builds, but gaps remain, 2025. URL https://web-assets.bcg.com/fd/0d/bcc5dfae4cbaa08c718b95b16cf5/ai-at-work-2025-slideshow-june-2025-edit-02.pdf.

Nghi DQ Bui. Building ai coding agents for the terminal: Scaffolding, harness, context engineering, and lessons learned. *arXiv preprint arXiv:2603.05344*, 2026.

Ira Ceka, Hailie Mitchell, Saurabh Pujar, Luca Buratti, Shyam Ramji, Junfeng Yang, Gail Kaiser, and Baishakhi Ray. Understanding automated program repair agents through the lens of traceability: An empirical study, 2026. URL https://arxiv.org/abs/2506.08311.

Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent readmes: An empirical study of context files for agentic coding, 2025. URL https://arxiv.org/abs/2511.12884.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Katsuki Chousa and Tsutomu Hirao. Automatic evaluation of language generation technology based on structure alignment. In *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 7663–7670, 2025.

Augment Code. Augment code, 2026a. URL https://www.augmentcode.com.

Augment Code. Augment code context engine, 2026b. URL https://www.augmentcode.com/context-engine.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL https://arxiv.org/abs/2509.16941.

Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. Mcptoolbench++: A large scale ai agent model context protocol mcp tool use benchmark, 2025. URL https://arxiv.org/abs/2508.07575.

Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. Configuring agentic ai coding tools: An exploratory study, 2026. URL https://arxiv.org/abs/2602.14690.

Pengfei Gao and Chao Peng. More with less: An empirical study of turn-control strategies for efficient coding agents, 2025. URL https://arxiv.org/abs/2510.16786.

Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. Trae agent: An llm-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.

Spandan Garg and Yufan Huang. Debug2fix: Supercharging coding agents with interactive debugging capabilities. *arXiv preprint arXiv:2602.18571*, 2026.

Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating agents. md: Are repository-level context files helpful for coding agents? *arXiv preprint arXiv:2602.11988*, 2026.

Aayush Gupta. Reliabilitybench: Evaluating llm agent reliability under production-like stress conditions. *arXiv preprint arXiv:2601.06112*, 2026.

Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. Agentic refactoring: An empirical study of ai coding agents, 2025. URL https://arxiv.org/abs/2511.04824.

Paul Jaccard. Étude comparative de la distribution florale dans une portion des alpes et des jura. *Bull Soc Vaudoise Sci Nat*, 37:547–579, 1901.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

Rick Kazman and Len Bass. Toward deriving software architectures from quality attributes. Technical Report CMU/SEI-94-TR-010, Software Engineering Institute, Carnegie Mellon University, Aug 1994. URL https://www.sei.cmu.edu/library/toward-deriving-software-architectures-from-quality-attributes/. Accessed: 2026-Aug-11.

Alon Lavie. Evaluating the output of machine translation systems. In *Proceedings of the 9th Conference of the Association for Machine Translation in the Americas: Tutorials*, 2010.

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T Barr, Federica Sarro, Zhaoyang Chu, and He Ye. Contextbench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*, 2026.

Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The swe-bench illusion: When state-of-the-art llms remember instead of reason, 2025. URL https://arxiv.org/abs/2506.12286.

Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. An empirical study on failures in automated issue solving, 2025a. URL https://arxiv.org/abs/2509.13941.

Yang Liu, Li Zhang, Fang Liu, Zhuohang Wang, Donglin Wei, Zhishuo Yang, Kechi Zhang, Jia Li, and Lin Shi. Reposcope: Leveraging call chain-aware multi-view context for repository-level code generation. *arXiv preprint arXiv:2507.14791*, 2025b.

Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M Zhang, Sebastian Baltes, and Christoph Treude. On the impact of agents. md files on the efficiency of ai coding agents. *arXiv preprint arXiv:2601.20404*, 2026.

Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. How to understand whole software repository. *arXiv preprint arXiv:2406.01422*, 2024.

Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. Understanding code agent behaviour: An empirical study of success and failure trajectories, 2025. URL https://arxiv.org/abs/2511.00197.

McKinsey. Superagency in the workplace: Empowering people to unlock ai's full potential, January 2025a. URL https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/superagency-in-the-workplace-empowering-people-to-unlock-ais-full-potential-at-work.

McKinsey. The state of ai, 2025b. URL https://www.mckinsey.com/~/media/mckinsey/business%20functions/quantumblack/our%20insights/the%20state%20of%20ai/2025/the-state-of-ai-how-organizations-are-rewiring-to-capture-value_final.pdf.

Microsoft. 2025: The year the frontier firm is born, 2025. URL https://www.microsoft.com/en-us/worklab/work-trend-index/2025-the-year-the-frontier-firm-is-born.

OpenAI. Harness engineering: leveraging codex in an agent-first world, 2026. URL https://openai.com/index/harness-engineering/.

OpenHands. The open platform for cloud coding agent. https://openhands.dev/, 2026.

Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. Repograph: Enhancing ai software engineering with repository-level code graph. *arXiv preprint arXiv:2410.14684*, 2024.

Dangfeng Pan, Zhensu Sun, Cenyuan Zhang, David Lo, and Xiaoning Du. The hidden cost of readability: How code formatting silently consumes your llm budget. *arXiv preprint arXiv:2508.13666*, 2025.

Yue Pan, Zimin Chen, Siyu Lu, Zhaoyang Chu, Xiang Li, Han Li, Yang Feng, Claire Le Goues, Federica Sarro, Martin Monperrus, and He Ye. Prometheus: Towards long-horizon codebase navigation for repository-level problem solving, 2026. URL https://arxiv.org/abs/2507.19942.

David Lorge Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

Dewayne E Perry and Alexander L Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software engineering notes*, 17(4):40–52, 1992.

Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. Towards a science of ai agent reliability. *arXiv preprint arXiv:2602.16666*, 2026.

Brian Randell. System structure for software fault tolerance. In *Proceedings of the international conference on Reliable software*, pp. 437–449, 1975.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*, 2020.

Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. Specrover: Code intent extraction via llms. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 963–974. IEEE, 2025.

Scarton Scarton, Mikel L Forcada, Miquel Espla-Gomis, and Lucia Specia. Estimating post-editing effort: a study on human judgements, task-based and reference-based metrics of mt quality. In *Proceedings of the 16th International Conference on Spoken Language Translation*, 2019.

Smolagents. Hugging face smolagents, 2026. URL https://huggingface.co/docs/smolagents/index.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li,

Xinhang Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Minqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence, 2026. URL https://arxiv.org/abs/2602.02276.

Boshi Wang, Weijian Xu, Yunsheng Li, Mei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. Improving code localization with repository memory, 2026a. URL https://arxiv.org/abs/2510.01003.

Jize Wang, Zerun Ma, Yining Li, Songyang Zhang, Cailian Chen, Kai Chen, and Xinyi Le. Gta: A benchmark for general tool agents. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 75749–75790. Curran Associates, Inc., 2024a. doi: 10.52202/079017-2412. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/8a75ee6d4b2eb0b777f549a32a5a5c28-Paper-Datasets_and_Benchmarks_Track.pdf.

Ruipeng Wang, Yuxin Chen, Yukai Wang, Chang Wu, Junfeng Fang, Xiaodong Cai, Qi Gu, Hui Su, An Zhang, Xiang Wang, Xunliang Cai, and Tat-Seng Chua. Agentnoisebench: Benchmarking robustness of tool-using llm agents under noisy condition, 2026b. URL https://arxiv.org/abs/2602.11348.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024b.

Yanlin Wang, Xinyi Xu, Jiachi Chen, Tingting Bi, Wenchao Gu, and Zibin Zheng. An empirical study of agent developer practices in ai agent frameworks, 2025a. URL https://arxiv.org/abs/2512.01939.

Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers, 2025b. URL https://arxiv.org/abs/2508.20453.

Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. On the use of agentic coding: An empirical study of pull requests on github, 2026. URL https://arxiv.org/abs/2509.14745.

Lloyd G Williams and Connie U Smith. Performance evaluation of software architectures. In *Proceedings of the 1st international workshop on Software and performance*, pp. 164–177, 1998.

Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint arXiv:2511.13646*, 2025.

Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. Reducing cost of llm agents with trajectory reduction. *Proceedings of the ACM on Software Engineering*, 3(FSE):1241–1263, 2026. ISSN 2994-970X. doi: 10.1145/3797084. URL http://dx.doi.org/10.1145/3797084.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. $\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL https://arxiv.org/abs/2406.12045.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, et al. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1592–1604, 2024.

# Appendix

## A Formal Definitions of Evaluation Metrics

We evaluate tool architectures along four complementary dimensions: *task resolve rate*, *consistency*, *exploration*, and *efficiency*.

**Task resolve rate.** For benchmark instance $i$ under setup $s$, let $c(i,s)$ denote the number of successful attempts out of $n$ total attempts. We define

$$\text{ResolveRate}(i,s) = \frac{c(i,s)}{n}. \tag{1}$$

Aggregating over instances gives the average probability that a randomly sampled attempt resolves the problem instance under setup $s$.

**Consistency.** Our main consistency metric is `pass^k`, which measures the probability that $k$ repeated attempts of the same problem instance are all successful. For benchmark instance $i$ under setup $s$, let $c(i,s)$ denote the number of passing attempts out of $n$ total attempts. We define

$$\texttt{pass\^{}k}(i,s) = \frac{\binom{c(i,s)}{k}}{\binom{n}{k}}. \tag{2}$$

Equivalently, this is the empirical proportion of all size-$k$ subsets of repeated attempts that fully pass (Chen et al., 2021). Under i.i.d. repeated attempts, it estimates the probability that $k$ attempts would all succeed. The metric equals 1 only when all attempts pass, and decreases as failures appear among reruns. In particular, `pass^1` reduces to the empirical pass rate, while larger $k$ imposes a stricter notion of repeatability by penalizing occasional failures more strongly. In our experiments, we use $k \in \{5, 7, 9\}$.

**Exploration.** We quantify exploration from two complementary perspectives. The first is *file read diversity*, which measures how differently repeated attempts traverse the repository. For each pair of attempts on the same problem instance, we compute the Jaccard distance between their sets of read files, and then average over attempt pairs and problem instances. Let $R_r(i,s)$ denote the set of files read in attempt $r$ for instance $i$ under setup $s$. We define

$$\text{ReadDiv}(i,s) = \frac{1}{\binom{n}{2}} \sum_{r_1<r_2} \left(1 - \frac{|R_{r_1}(i,s) \cap R_{r_2}(i,s)|}{|R_{r_1}(i,s) \cup R_{r_2}(i,s)|}\right). \tag{3}$$

Larger values indicate that repeated attempts tend to inspect more different subsets of the repository.

The second exploration metric is *solution diversity*. Even when attempts read different files, they may still converge to nearly identical outputs. We therefore measure the mean pairwise CodeBLEU distance between generated solutions across repeated attempts of the same problem instance. Let $\text{CodeBLEU}(y_{r_1}, y_{r_2})$ denote the similarity between two generated solutions. We define

$$\text{CodeBLEU-Dist}(i,s) = \frac{1}{\binom{n}{2}} \sum_{r_1<r_2} \left(1 - \text{CodeBLEU}(y_{r_1}(i,s), y_{r_2}(i,s))\right). \tag{4}$$

Larger values indicate that repeated attempts produce more diverse final solutions.

**Efficiency.** We measure efficiency at the trajectory level. A *trajectory* $\tau$ is a complete interaction sequence for one attempt on one benchmark instance:

$$\tau = (Q, A_1, O_1, A_2, O_2, \ldots, A_T, O_T), \tag{5}$$

where $Q$ is the initial task prompt, $A_t$ is the action issued by the actor at step $t$, $O_t$ is the resulting environment observation, and $T$ is the total number of interaction steps before termination.

The *trajectory length* is the number of interaction steps:

$$\text{Steps}(\tau) = T. \tag{6}$$

Table 3: Atomic tools and their bash equivalents.

| Atomic tool | Purpose | Equivalent bash command(s) |
|---|---|---|
| `search` | Search repository contents | `grep -R`, `find`, `rg` |
| `view` | Read a bounded file region | `sed -n`, `head`, `tail` |
| `str_replace` | Replace a targeted string | `sed -i`, `perl -0pi -e` |
| `create` | Create a new file | `cat > file`, heredoc, `printf > file` |
| `bash` | Execute arbitrary shell commands | `bash` |

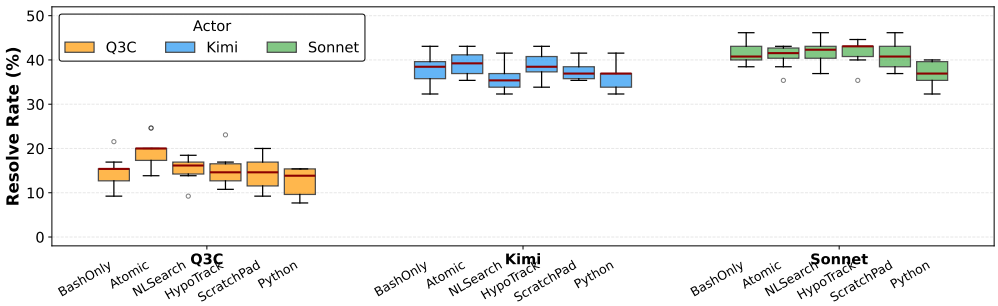



Figure 5: Overall resolve rates across actors and tool architectures. Results are broadly similar across setups, reflecting our design choice to keep tool capability close to the BashOnly baseline and vary primarily the architecture of the interface.

We define *input token cost* as the total number of tokens sent to the model across all API calls in the trajectory. Because each API call includes the original task together with the accumulated interaction history, this quantity is cumulative across steps:

$$\text{InputTok}(\tau) = \sum_{t=1}^{T} |(Q, A_1, O_1, \ldots, A_{t-1}, O_{t-1})| \,. \tag{7}$$

We define *output token cost* as the total number of tokens generated by the model across all steps:

$$\text{OutputTok}(\tau) = \sum_{t=1}^{T} |A_t|. \tag{8}$$

We additionally track *observation tokens*, defined as the total number of tokens returned by the environment across all steps:

$$\text{ObsTok}(\tau) = \sum_{t=1}^{T} |O_t|. \tag{9}$$

# B Additional Discussions for Evaluation Results

## B.1 Performance

Figure 5 shows that overall performance remains in a similar range across tool architectures. This is expected, since we intentionally control tool capability to stay close to the BashOnly setup and vary primarily the way those capabilities are organized and exposed.

## B.2 Consistency Results with Additional Repeats

Some of the consistency improvements from `Atomic` in the main paper, especially for `Kimi-K2.5`, are numerically modest. To check whether these gains remain stable under

Table 4: Consistency robustness under additional repeats for open-weight actors. We compare `BashOnly` and `Atomic` using 10 and 30 repeated rollouts per instance. For each metric, Δ denotes `Atomic` − `BashOnly`.

| Actor | Repeats | pass^5 | | | pass^7 | | | pass^9 | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | Bash | Atomic | Δ | Bash | Atomic | Δ | Bash | Atomic | Δ |
| `Qwen3Coder-30B` | 10 | 0.046 | 0.106 | **+0.059** | 0.031 | 0.097 | **+0.067** | 0.020 | 0.094 | **+0.074** |
| | 30 | 0.044 | 0.111 | **+0.067** | 0.033 | 0.097 | **+0.064** | 0.027 | 0.087 | **+0.060** |
| `Kimi-K2.5` | 10 | 0.290 | 0.304 | **+0.014** | 0.277 | 0.289 | **+0.013** | 0.266 | 0.280 | **+0.014** |
| | 30 | 0.303 | 0.318 | **+0.015** | 0.292 | 0.307 | **+0.015** | 0.283 | 0.300 | **+0.016** |

more repeated sampling, we rerun the `BashOnly`–`Atomic` comparison with a larger number of rollouts per instance and recompute `pass^5`, `pass^7`, and `pass^9`.

Because this analysis substantially increases evaluation cost, we limit it to the two open-weight actors: `Qwen3Coder-30B` and `Kimi-K2.5`. In addition to the main-paper setting with 10 repeats, we also evaluate a 30-repeat variant while keeping the benchmark instances and protocol otherwise unchanged.

Table 4 shows that the main result is stable. For `Qwen3Coder-30B`, `Atomic` improves all three consistency metrics at both repeat counts, with gains of $+0.059$, $+0.067$, and $+0.074$ at 10 repeats, and $+0.067$, $+0.064$, and $+0.060$ at 30 repeats. For `Kimi-K2.5`, the effect is smaller but remains uniformly positive, increasing from $+0.014$, $+0.013$, and $+0.014$ at 10 repeats to $+0.015$, $+0.015$, and $+0.016$ at 30 repeats. These results suggest that the consistency advantage of `Atomic` is robust to substantially more repeated evaluation, rather than a small-sample fluctuation from the 10-repeat setting.

### B.3 Mechanism Analysis: Atomic Reduces Environment-Interaction Errors

To help explain why `Atomic` yields the clearest consistency gains, we examine environment-interaction errors as a possible mechanism. This analysis is motivated by prior work in software dependability Avizienis et al. (2004), which analyzes unreliability through a fault–error–failure chain, and by recent work on agent reliability Gupta (2026); Rabanser et al. (2026), which studies repeated execution together with robustness to tool and API failures. Following this perspective, we treat environment-interaction errors as a plausible lens for understanding why some tool architectures lead to lower repeated-run reliability than others.

We focus on the comparison between `BashOnly` and `Atomic`, because `Atomic` is the only setup that improves `pass^k` for all three actors and yields the largest gains for `Qwen3Coder-30B`. Figure 6 shows that these consistency differences are accompanied by corresponding differences in environment-interaction errors. Under `BashOnly`, `Qwen3Coder-30B` incurs 3.11 interaction errors per trajectory on average, compared with 0.35 for `Kimi-K2.5` and 0.54 for `Sonnet-4.5`. This ordering mirrors the consistency results in Table 2: the actor with the lowest `pass^k` values also exhibits the highest baseline rate of environment-interaction errors.

To examine this pattern more closely, we divide environment-interaction errors into three categories (the "Error Categories" subplot in Figure 6). *Misaligned-param* refers to cases where the agent applies a tool to the wrong target, such as a nonexistent file path, an incorrect working directory, or an invalid runtime target. *Mis-edit* refers to editing mistakes that leave the program in a broken state, such as introducing `SyntaxError`, `IndentationError`, or other execution-breaking edit failures. *Wrong-syntax* refers to malformed tool invocations, such as invalid shell, `sed`, or `awk` expressions, or an incorrect binary name.

The category breakdown suggests that the reduction is concentrated in the error types most directly targeted by a structured interface. For `Qwen3Coder-30B`, `Atomic` reduces *mis-edit* errors from 1.64 to 0.19 and *wrong-syntax* errors from 0.96 to 0.01. These are precisely

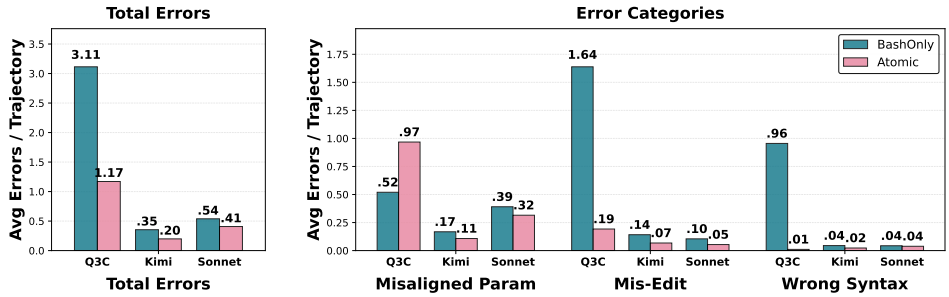



Figure 6: `Atomic` is associated with fewer environment-interaction errors, especially for weaker actors. The figure shows the average number of environment-interaction errors per trajectory under `BashOnly` and `Atomic`, both in total and broken down by error category for each actor. The largest reduction appears for `Qwen3Coder-30B`, which also benefits most from `Atomic` in consistency.

the failure modes that should become less common when free-form shell editing and command construction are replaced by narrower, constrained operations. For `Kimi-K2.5` and `Sonnet-4.5`, the corresponding reductions are smaller, largely because these error categories are already rare under `BashOnly`. This is consistent with the smaller consistency gains of `Atomic` for the stronger actors.

One exception is *misaligned-param*. For `Qwen3Coder-30B`, this category increases under `Atomic`, from 0.52 to 0.97. A plausible explanation is that `Atomic` enforces stricter parameter validation than unconstrained bash commands. Under this interpretation, some mistakes that might previously have failed silently, partially applied, or gone unnoticed under `BashOnly` are instead surfaced explicitly as parameter-alignment errors. The increase therefore does not necessarily imply worse high-level localization; it may instead reflect more explicit detection and typing of the same underlying mistakes. Even with this increase, the large reductions in *mis-edit* and *wrong-syntax* dominate the overall change in Qwen's total error count.

Taken together, the error analysis is consistent with the interpretation that `Atomic`'s consistency gains arise in part from reducing low-level environment-interaction errors, especially for actors that are more prone to such errors under `BashOnly`. Note that this analysis does not directly test the causal link between environment-interaction errors and task-level inconsistency. Instead, we follow prior work in software dependability and agent reliability in treating such errors as a plausible explanatory lens for repeated-run reliability.

### B.4 Why NLSearch Broadens Exploration and What It Covers

**NLSearch produces the strongest gains in repository exploration, while effects on downstream solution diversity remain limited.** Figure 3 in the main paper shows that `NLSearch` is the only setup that consistently increases *read diversity* across all three actors. The gains are substantial for all models, whereas most other setups have smaller or more actor-specific effects. By contrast, *solution diversity* changes much less. Across most actor–setup pairs, repeated attempts under different tool architectures still converge to similar final patches even when they inspect different parts of the repository. Thus, the effect of tool architecture is more pronounced on repository traversal than on downstream solution behavior.

A notable exception is `Atomic` for `Qwen3Coder-30B`, which sharply reduces solution diversity. This aligns with the consistency analysis (Section 3.2): `Atomic` substantially reduces Qwen's *mis-edit* and *wrong-syntax* errors, removing a major source of accidental patch variation. With less broken or noisy editing, repeated attempts converge to a narrower set of cleaner final patches.

**A likely reason is that NLSearch induces more diverse early search actions.** To better understand why `NLSearch` broadens repository exploration, we next examine how search is

Table 5: Diversity of early search actions under `NLSearch` and `BashOnly`. We extract search actions from the first 20 steps of each rollout and compare them across repeated runs of the same benchmark instance. For `NLSearch`, actions are natural-language retrieval queries; for `BashOnly`, they are search patterns issued through `grep` and `find`. We report Jaccard distance and normalized Levenshtein distance over these strings. Higher values indicate more diverse search formulations.

| Actor | Setup | Jaccard | Levenshtein |
|---|---|---|---|
| `Qwen3Coder-30B` | NLSearch | 0.850 | 0.579 |
| | BashOnly | 0.692 | 0.486 |
| `Kimi-K2.5` | NLSearch | 0.873 | 0.563 |
| | BashOnly | 0.634 | 0.451 |
| `Sonnet-4.5` | NLSearch | 0.862 | 0.521 |
| | BashOnly | 0.681 | 0.447 |

Table 6: Manual validation of the high-relevant-file proxy. We compare the proxy label against human judgments of whether each accessed file is helpful for resolving the issue.

| | Human Helpful | Human Not Helpful |
|---|---|---|
| Proxy Relevant | 561 | 33 |
| Proxy Not Relevant | 14 | 11 |

performed early in a trajectory. It is worth noting that `NLSearch` is implemented as a search sub-agent using the same base LLM as the actor. We do not rely on additional program-analysis structures (Liu et al., 2025b; Ouyang et al., 2024; Ma et al., 2024) or semantic embeddings (Code, 2026b) to support natural-language search. As a result, the observed effects are best understood as coming from how the search interface is provided, rather than from additional underlying information. Qualitatively, actors under `BashOnly` often repeat similar `grep`/`find` patterns across repeated attempts, whereas under `NLSearch` they issue more varied natural-language queries.

To quantify this effect, we compare `BashOnly` and `NLSearch` during early exploration. We extract search actions from the first 20 steps of each rollout as a proxy for initial repository search. For `NLSearch`, these actions are natural-language retrieval queries; for `BashOnly`, they are the search strings passed to `grep` and `find`. Table 5 shows that `NLSearch` yields higher diversity under both Jaccard distance and normalized Levenshtein distance for all three actors. This suggests that `NLSearch` broadens repository traversal by inducing more diverse search formulations early in the trajectory.

**The additional exploration induced by NLSearch also covers more relevant files, but with lower precision.** Broader exploration is only useful if the additional files accessed are actually relevant to the problem instance. To assess this, we introduce a conservative file-level proxy. For each benchmark instance, a file is labeled *high-relevant* if it is read by at least three successful trajectories from *every* setup. This definition focuses on files that recur across successful resolutions under diverse tool architectures, while reducing bias toward any single setup.

**Manual validation of the high-relevant-file proxy.** Because the high-relevant-file set is a proxy rather than a human-labeled gold standard, we additionally validate it with a small manual annotation study. We sample 30 trajectories and label 622 file operations according to whether the accessed file is helpful for resolving the corresponding issue. The annotation protocol is LLM-assisted: we first use Codex to generate reference reasoning for each file access, and then ask human annotators to judge the final helpfulness label. We exclude 3 cases where the human judgment and the reference reasoning diverge, leaving 619 file operations for evaluation.

Table 7: Quality of explored context under different tool architectures. For each actor–setup pair, we compare the files read during a rollout against a conservative set of *high-relevant* files for that benchmark instance. A file is labeled high-relevant if it is read by at least three successful trajectories from every setup. Values in parentheses show the difference relative to `BashOnly` for the same actor. `NLSearch` consistently improves recall, but at the cost of lower precision.

| Actor | Setup | Precision | Recall |
|---|---|---|---|
| `Qwen3Coder-30B` | BashOnly | 0.938 | 0.406 |
| | Atomic | **0.944 (+0.006)** | 0.431 (+0.025) |
| | NLSearch | 0.891 (-0.046) | **0.452 (+0.046)** |
| | HypoTrack | 0.938 (+0.001) | 0.418 (+0.012) |
| | ScratchPad | 0.932 (-0.006) | 0.410 (+0.004) |
| | Python | 0.888 (-0.050) | 0.365 (-0.042) |
| `Kimi-K2.5` | BashOnly | 0.939 | 0.548 |
| | Atomic | 0.902 (-0.037) | 0.571 (+0.023) |
| | NLSearch | 0.844 (-0.095) | **0.601 (+0.053)** |
| | HypoTrack | **0.950 (+0.011)** | 0.555 (+0.007) |
| | ScratchPad | 0.946 (+0.007) | 0.549 (+0.001) |
| | Python | 0.937 (-0.002) | 0.529 (-0.019) |
| `Sonnet-4.5` | BashOnly | 0.919 | 0.578 |
| | Atomic | 0.915 (-0.004) | 0.588 (+0.010) |
| | NLSearch | 0.816 (-0.104) | **0.643 (+0.064)** |
| | HypoTrack | 0.927 (+0.008) | 0.563 (-0.016) |
| | ScratchPad | 0.932 (+0.013) | 0.564 (-0.014) |
| | Python | **0.946 (+0.027)** | 0.532 (-0.047) |

Table 6 shows that the high-relevant-file proxy aligns well with human helpfulness judgments in this sampled validation set. The proxy achieves 94.4% precision and 97.6% recall:

$$\text{Precision} = \frac{561}{561+33} = 94.4\%, \qquad \text{Recall} = \frac{561}{561+14} = 97.6\%.$$

This result supports the use of the proxy as a conservative approximation to helpful file access. In particular, it suggests that the proxy is not merely capturing files that are frequently read under broader-exploration setups such as NLSearch, but is generally aligned with files humans judge as useful for resolving the task. At the same time, we treat this validation as supporting evidence rather than a perfect ground truth, since file usefulness can depend on trajectory context and may include files that help reasoning without appearing in the final patch.

**Table 7 evaluates the files read under each setup against this high-relevance set.** `NLSearch` achieves the highest recall for all three actors, indicating that it covers more files plausibly needed for successful resolution. At the same time, its precision is lower than the `BashOnly` baseline. In other words, `NLSearch` broadens exploration in a meaningful sense by improving coverage of relevant files, but it also introduces additional context that the actor must filter and use effectively.

### B.5 Detailed Efficiency Analysis and Compound Interaction

**The efficiency advantage of `Python` mainly comes from reducing interaction steps.** Compared with `BashOnly`, `Python` substantially shortens trajectories for all three actors. At the same time, its total output tokens and total observation tokens remain broadly similar rather than uniformly smaller. This suggests that the main efficiency gain does not come from making each interaction cheaper, but from reducing the number of API calls needed to complete an attempt.

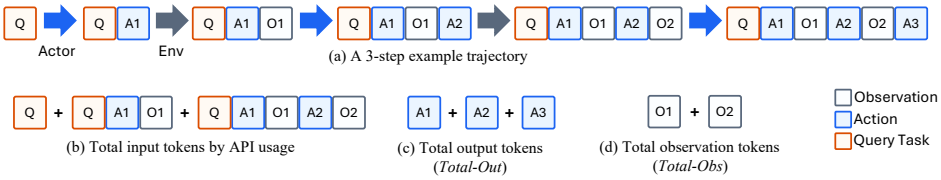



Figure 7: Illustration of the token metrics used in Table 8. The decomposition clarifies why trajectory length strongly affects cumulative input cost: each additional interaction step requires another API call, which includes both the new action and the cumulative interaction history. Panel (a) shows a three-step trajectory with an initial query task $Q$, actions $A_t$, and observations $O_t$. Panel (b) illustrates cumulative input tokens as the total tokens sent to the model across API calls, including the repeated query context and the growing history of prior actions and observations. Panel (c) shows total output tokens, defined as the sum of all action tokens generated by the model. Panel (d) shows total observation tokens, defined as the sum of all environment outputs returned to the model.

Table 8: Efficiency comparison between `BashOnly` and `Python`. Each cell reports the mean value of the corresponding trajectory-level metric.

| Actor | Setup | Steps | Total-Out | Total-Obs | Out/Step | Obs/Step |
|---|---|---|---|---|---|---|
| `Qwen3Coder-30B` | `BashOnly` | 77 | 15,535 | 15,228 | 197 | 230 |
| | `Python` | 46 | 12,854 | 15,992 | 315 | 410 |
| `Kimi-K2.5` | `BashOnly` | 65 | 13,087 | 23,670 | 196 | 389 |
| | `Python` | 47 | 11,288 | 26,298 | 226 | 607 |
| `Sonnet-4.5` | `BashOnly` | 80 | 27,168 | 32,707 | 355 | 438 |
| | `Python` | 55 | 18,082 | 28,915 | 334 | 561 |

Figure 7 clarifies this distinction. Total input tokens accumulate across API calls, so longer trajectories repeatedly pay for the query context together with a growing history of prior actions and observations. By contrast, total output tokens and total observation tokens measure the amount of generated action content and returned environment output, respectively. As a result, an interface can reduce overall cost mainly by shortening the trajectory, even if the amount of work performed in each individual step stays similar or becomes larger.

Table 8 is consistent with this explanation. Across all three actors, `Python` reduces interaction steps substantially relative to `BashOnly`: from 77 to 46 for `Qwen3Coder-30B`, from 65 to 47 for `Kimi-K2.5`, and from 80 to 55 for `Sonnet-4.5`. For `Qwen3Coder-30B` and `Kimi-K2.5`, total output and total observation tokens remain broadly similar under the two setups, while per-step output and observation tokens are often larger under `Python`. This indicates that the actor is carrying out more work in each interaction. The resulting efficiency gain therefore comes primarily from step reduction and the associated reduction in cumulative input cost.

**For `Sonnet-4.5`, the efficiency gain of `Python` comes not only from fewer steps, but also from reducing speculative full-file rewrites through faster execution feedback.** With `BashOnly`, we often observe consecutive full-file rewrites to the same file, where the agent repeatedly revises content before executing or validating it. This resembles a form of *speculative revision*: the agent keeps rewriting the file without receiving feedback that would guide the next change. Under `Python`, this behavior is much less frequent, likely because editing and execution are more naturally coupled within a single interaction, so the agent receives faster feedback and makes more directed modifications.

Figure 8 illustrates this effect. For each trajectory, we measure the number of tokens spent on consecutive writes to the same file before a validating action, and plot the complementary cumulative distribution across trajectories. The curve for `Python` lies consistently below that of `BashOnly`, showing that `Python` produces fewer trajectories with large amounts of waste from speculative revision. This reduction is especially pronounced in the high-waste

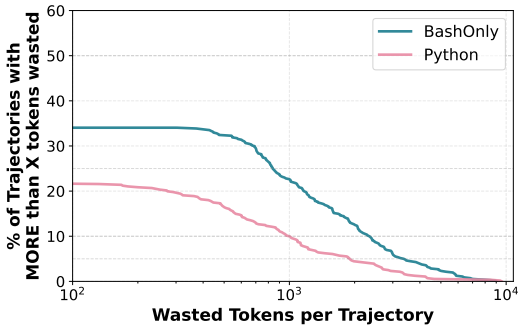



Figure 8: Wasted tokens from speculative revision under `BashOnly` and `Python` for Sonnet-4.5. For each trajectory, we measure tokens spent on consecutive writes to the same file before a validating action. The complementary cumulative distribution shows that `Python` substantially reduces these wasted tokens, especially in the high-waste tail.

Table 9: Estimated compounding of `BashOnly` trajectories, measured by parsing each bash step into atomic-style operations. Higher `Ops/Step` indicates that a single bash step combines more underlying operations.

| Actor | READ | SEARCH | EDIT | WRITE | RUN | GIT | MISC | Ops/Step |
|---|---|---|---|---|---|---|---|---|
| `Qwen3Coder-30B` | 18.1 | 14.1 | 7.6 | 5.3 | 17.2 | 2.0 | 24.1 | 1.154 |
| `Kimi-K2.5` | 30.0 | 10.9 | 0.5 | 8.2 | 20.6 | 2.5 | 11.1 | 1.403 |
| `Sonnet-4.5` | 32.3 | 18.6 | 0.5 | 18.1 | 30.5 | 3.4 | 12.3 | 1.500 |

tail, suggesting that part of `Python`'s efficiency gain for `Sonnet-4.5` comes from avoiding repeated unguided rewrites rather than only shortening the number of steps.

**The effect of `Atomic` depends on how compound the actor's original bash interaction is.** If an actor already combines multiple low-level operations into a single bash step, then replacing bash with `Atomic` can split that work across more turns and increase overhead. If the actor is already closer to an atomic interaction style under `BashOnly`, the same interface imposes less fragmentation and may even improve efficiency.

Table 10 shows that this ordering closely matches the efficiency effect of replacing `BashOnly` with `Atomic`. For `Sonnet-4.5` and `Kimi-K2.5`, which show more compound bash behavior, `Atomic` increases both steps and input tokens. By contrast, `Qwen3Coder-30B`, which is closer to an atomic interaction style under `BashOnly`, becomes more efficient under `Atomic`. For `Qwen3Coder-30B`, however, the benefit of reducing low-level interaction errors appears to outweigh the overhead of a more fine-grained interface; see Appendix B.3. This pattern suggests that the efficiency effect of tool architecture depends not only on what operations the interface makes available, but also on whether its granularity matches the actor's natural interaction style.

**Python does not introduce new action capabilities.** Finally, we manually inspect 100 Python actions sampled from different actors and trajectories to check whether the Python setup changes the available actions, rather than only the interaction style. For each action, we identify whether the same operation can be carried out in BashOnly, with Codex assisting the mapping. Table 11 summarizes the result.

Overall, 97 of the 100 inspected Python actions correspond to operations already available in BashOnly. Note that the 8 test/check scripts do not add new action capabilities, since `BashOnly` actors commonly create the same script with a heredoc and execute it with `python3`. This supports our interpretation that Python improves efficiency by changing how actors express compound interactions, rather than by introducing new repository information or new actions.

Table 10: Effect of replacing `BashOnly` with `Atomic`. Actors with more compound bash behavior tend to incur more step and token overhead under atomic tools.

| Actor | Ops/Step | Steps (bash→atomic) | Input Tokens (bash→atomic) | Output Tokens (bash→atomic) |
|---|---|---|---|---|
| `Qwen3Coder-30B` | 1.15 | 64 → 43$_{\downarrow 33\%}$ | 980K → 662K$_{\downarrow 32\%}$ | 11444 → 9672$_{\downarrow 15\%}$ |
| `Kimi-K2.5` | 1.40 | 56 → 59$_{\uparrow 5\%}$ | 950K → 1141K$_{\uparrow 20\%}$ | 9556 → 9659$_{\uparrow 1\%}$ |
| `Sonnet-4.5` | 1.50 | 74 → 80$_{\uparrow 8\%}$ | 2140K → 2576K$_{\uparrow 20\%}$ | 25357 → 25089$_{\downarrow 1\%}$ |

Table 11: Manual inspection of whether Python actions introduce capabilities beyond BashOnly.

| Count | BashOnly counterpart |
|---|---|
| 89 | Directly correspond to operations available in `BashOnly` |
| 8 | Python test/check scripts that `BashOnly` commonly create and run from bash |
| 3 | Broken-syntax Python code whose intended operation is unclear |

### B.6 How Tool-Architecture Effects Generalize to More Coding Tasks

We study how the main tool-architecture effects generalize beyond our SWE-bench Live (Zhang et al., 2025) subset by evaluating additional coding tasks on 72 instances from 23 repositories across issue resolving, feature implementation, and debugging. Specifically, we use SWE-bench Verified (Jimenez et al., 2023) for issue resolving and SWE-bench Pro (Deng et al., 2025) for feature implementation. For debugging, we construct a separate SWE-bench split by selecting instances that contain full stack traces in the issue description.

Due to budget constraints, we run this study with the two open-weight actors, `Qwen3Coder-30B` and `Kimi-K2.5`. We do not repeat the full main evaluation. Instead, this validation focuses on the pairwise comparisons most directly tied to our main findings: `Atomic` vs. `BashOnly` for consistency, `NLSearch` vs. `BashOnly` for exploration, and `Python` vs. `BashOnly` for efficiency. For each actor, task, and setup, we run 5 independent rollouts per instance. Tables 12–15 report the results.

Overall, the additional validation supports the main findings beyond the original SWE-bench Live subset. Table 12 shows that `Atomic` improves `pass^5` for both actors overall. Table 13 shows that `NLSearch` increases read diversity across tasks. Tables 14 and 15 show that `Python` generally reduces both input-token cost and trajectory length.

## C Why Cognitive Scaffolding Tools Have Limited Effects

The two cognitive-scaffolding setups, `Scratchpad` and `HypoTrack`, have only limited effects in our main evaluation. We interpret this result as specific to the lightweight text-based scaffolds studied here. These tools provide an explicit channel for recording intermediate reasoning, but they do not add retrieval, memory management, new task information, or enforce a different reasoning policy. Thus, they mainly test whether a simple text interface for actor-written intermediate state changes behavior under otherwise similar capabilities. In our setting, actors usually do not use these tools in ways that substantially change their reasoning: under `Scratchpad`, actors often produce content that closely matches reasoning already present in `BashOnly`; under `HypoTrack`, actors rarely maintain multiple competing hypotheses. Taken together, these results suggest that lightweight text-based cognitive scaffolding alone is insufficient to reliably alter actor reasoning behavior.

**Scratchpad mostly repeats reasoning that already appears under `BashOnly`.** To test whether `Scratchpad` introduces genuinely new reasoning, we compare scratchpad content against the reasoning content produced in the corresponding `BashOnly` trajectories. For each scratchpad entry, we compute its BLEU score relative to the original `BashOnly` reasoning content. Figure 9 shows that these scores are generally high across all three actors, with many entries

Table 12: Effect of Atomic on consistency across coding tasks. Each cell reports Δpass^5, computed as Atomic minus BashOnly, in percentage points. Positive values indicate higher consistency under Atomic.

| Actor | SWE-bench Live | SWE-bench Verified | Feature Impl. | Debugging | Overall |
|---|---|---|---|---|---|
| Qwen3Coder-30B | +5.9 | +15.6 | +8.4 | +8.0 | +9.5 |
| Kimi-K2.5 | +1.4 | +4.4 | +4.2 | +0.0 | +2.5 |

Table 13: Effect of NLSearch on exploration across coding tasks. Each cell reports the relative change in read diversity, computed as NLSearch relative to BashOnly. Positive values indicate higher read diversity under NLSearch.

| Actor | SWE-bench Live | SWE-bench Verified | Feature Impl. | Debugging | Overall |
|---|---|---|---|---|---|
| Qwen3Coder-30B | +18.6% | +23.3% | +2.7% | +7.5% | +13.0% |
| Kimi-K2.5 | +21.3% | +13.0% | +0.6% | +2.7% | +9.4% |

above 0.6. This indicates that scratchpad content is often textually close to reasoning the model would already have produced without the tool (Lavie, 2010; Chousa & Hirao, 2025; Scarton et al., 2019). In other words, the scratchpad frequently acts as an explicit channel for restating existing reasoning, rather than as a tool that elicits substantially different intermediate thoughts or planning structure.

**HypoTrack rarely induces multi-branch reasoning.** The intended role of HypoTrack is to encourage the model to manage alternative debugging hypotheses explicitly. To evaluate whether this happens in practice, we classify trajectories into three cases: *no hypothesis*, where the tool is never called; *single hypothesis*, where only one hypothesis is recorded without branching; and *branching hypotheses*, where the model records multiple alternative hypotheses for the same problem instance. Figure 10 shows that true branching behavior is rare. Most trajectories either do not use the tool at all or use it in a single-threaded way by recording only one hypothesis. Thus, although HypoTrack provides an interface for structured branch management, actors seldom use it to explore multiple competing explanations.

## D How Actors Use the Provided Tools Across Architectures

Tool usage differs substantially across architectures. Figure 11 shows the number of calls per trajectory for each tool setup, while Figure 12 shows when those calls occur within a trajectory, measured as the step of a tool call divided by the total trajectory length. Two patterns are clear. First, actors adapt strongly to Atomic and Python: these tools are used frequently across all three actors, indicating that they become part of the main execution loop rather than serving as occasional auxiliary tools. By contrast, NLSearch is used much less frequently, which is consistent with its role as an exploration tool rather than a general interaction interface. The cognitive-scaffolding tools are also used relatively sparingly: HypoTrack is called fewer than five times in most trajectories, and Scratchpad is called fewer than ten times in most trajectories for Kimi-K2.5 and Sonnet-4.5.

Figure 12 further shows that Atomic and Python calls are distributed relatively evenly across the full trajectory, consistent with their role in routine interaction. One exception is that Atomic calls become slightly less frequent late in the trajectory, likely because actors often switch back to bash for test execution (which needs to execute the python bash command). NLSearch concentrates near the beginning of trajectories, which matches its role in repository exploration. HypoTrack calls cluster near the beginning and the end: the beginning typically corresponds to recording an initial hypothesis, while the end often corresponds to updating the status of that same hypothesis. This pattern suggests that actors seldom use HypoTrack to maintain multiple competing lines of reasoning, which is consistent with the analysis in Appendix C. Overall, these results suggest that actors do adapt their tool use to the

Table 14: Effect of `Python` on input-token efficiency across coding tasks. Each cell reports Δ input tokens, computed as `Python` minus `BashOnly`, in millions. Negative values indicate lower input-token cost under `Python`.

| Actor | SWE-bench Live | SWE-bench Verified | Feature Impl. | Debugging | Overall |
|---|---|---|---|---|---|
| `Qwen3Coder-30B` | -0.8 | -0.6 | -1.1 | +0.2 | -0.6 |
| `Kimi-K2.5` | -0.3 | +0.0 | -1.1 | -0.2 | -0.4 |

Table 15: Effect of `Python` on step efficiency across coding tasks. Each cell reports Δ steps, computed as `Python` minus `BashOnly`. Negative values indicate shorter trajectories under `Python`.

| Actor | SWE-bench Live | SWE-bench Verified | Feature Impl. | Debugging | Overall |
|---|---|---|---|---|---|
| `Qwen3Coder-30B` | -31.4 | -25.0 | -40.3 | -6.9 | -25.9 |
| `Kimi-K2.5` | -17.7 | -8.7 | -25.0 | -8.8 | -15.1 |

provided architecture, but the degree and manner of adaptation differ substantially across tools.

## E Detailed Discussion of Related Work

**Empirical studies of coding agents.** A growing body of recent work empirically studies coding agents, but mostly from perspectives other than tool architecture. One line of work is problem-focused, analyzing where coding agents succeed or fail on tasks such as issue resolution, test generation, and patch construction. Prior studies examine agent trajectories and error patterns on software engineering benchmarks, analyze patch quality and failure modes, and question whether strong benchmark results always reflect genuine reasoning rather than memorization (Ceka et al., 2026; Majgaonkar et al., 2025; Liang et al., 2025). A second line studies how developers use coding agents in practice, including empirical analyses of agent-assisted refactoring, developer adoption patterns, and broader engineering practices around agent frameworks (Horikawa et al., 2025; Watanabe et al., 2026; Wang et al., 2025a). A third line focuses more on planning and failure behavior, for example by characterizing how agent-based systems break down during automated issue solving (Liu et al., 2025a). Finally, recent work also studies agent context and configuration, such as how developers provide contextual instructions, configure coding agents, or maintain agent-oriented repository files (Galster et al., 2026; Chatlatanagulchai et al., 2025).

Although these studies provide important empirical evidence about coding-agent behavior, they do not consider tool architecture as a design variable. As a result, they offer limited guidance on how the organization and exposure of tools themselves shape agent behavior.

**Empirical studies of tool design.** Another related line of work studies tool use more directly, especially whether agents can identify and invoke the appropriate tool in complex environments. Benchmarks such as MCP-Bench (Wang et al., 2025b) and related work evaluate tool-using agents under large toolsets, realistic APIs, and multi-step tasks, emphasizing challenges in tool retrieval, schema interpretation, and invocation accuracy (Fan et al., 2025). However, this line of work primarily treats tool design as a question of capability and correct invocation. In contrast, our work asks whether, even when underlying capabilities are similar, differences in interface architecture, namely how capabilities are organized and exposed, can systematically influence agent behavior.

**Agent non-functional properties.** Recent work has also begun to examine non-functional properties of agents, moving beyond aggregate task success. For example, prior work studies efficiency, such as how turn-control strategies affect the cost of coding agents (Gao & Peng, 2025); robustness, such as how tool-using agents behave under noisy or perturbed environments (Wang et al., 2026b); and reliability, including broader calls for more systematic evaluation of consistency and dependability in agentic systems (Rabanser et al., 2026).

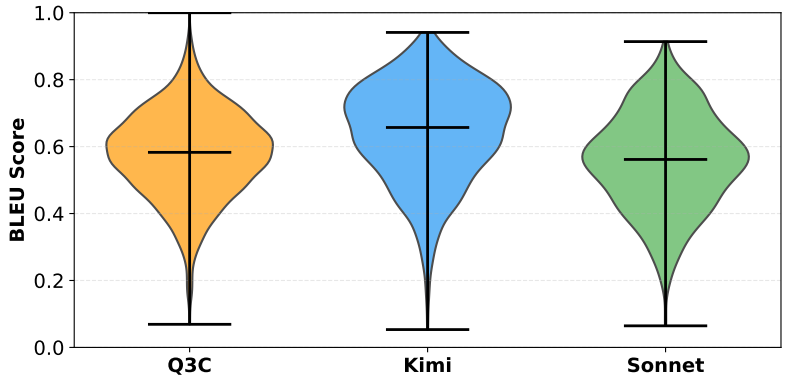



Figure 9: Scratchpad entries are highly similar to reasoning content that already appears in `BashOnly`. For each scratchpad entry, we compute its BLEU score against the corresponding reasoning content from the `BashOnly` setup. Across all three actors, the distribution is concentrated at relatively high scores, indicating that the scratchpad often externalizes reasoning that the model would have produced anyway rather than introducing substantively new intermediate state.

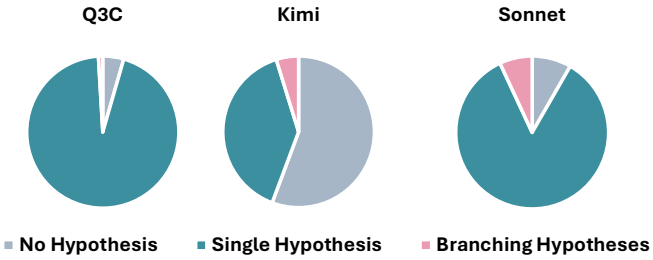



Figure 10: Hypothesis tracking rarely induces genuine branching behavior. We categorize trajectories into three cases: no hypothesis-tracker use, single-hypothesis use, and branching-hypothesis use. Across all three actors, most trajectories either never use the tool or record only a single hypothesis, while true branching over multiple alternative hypotheses is rare.

These works establish the importance of non-functional properties, but they do not connect them to specific tool-architecture choices. Our study addresses this gap by examining how tool architecture affects efficiency, repeated-attempt consistency, and exploration.

**Tool design in coding agents.** A substantial body of work on coding agents proposes concrete tool designs, including repository navigation tools, structured search interfaces, semantic retrieval systems, and agent frameworks built around different interaction abstractions (Pan et al., 2026; Yang et al., 2024; Liu et al., 2025b; Ouyang et al., 2024; Anthropic, 2026c; Antoniades et al., 2024; OpenHands, 2026). These systems are directly relevant to our work because they motivate the representative tool architectures considered in our study.

However, they typically introduce new tools as part of end-to-end agent designs, often changing capability and interface at the same time. As a result, although they show that particular tool designs can improve task performance, they do not systematically isolate the effect of tool architecture under similar capabilities. Our work instead treats tool architecture itself as an experimental variable and studies its effect on efficiency, consistency, and exploration.

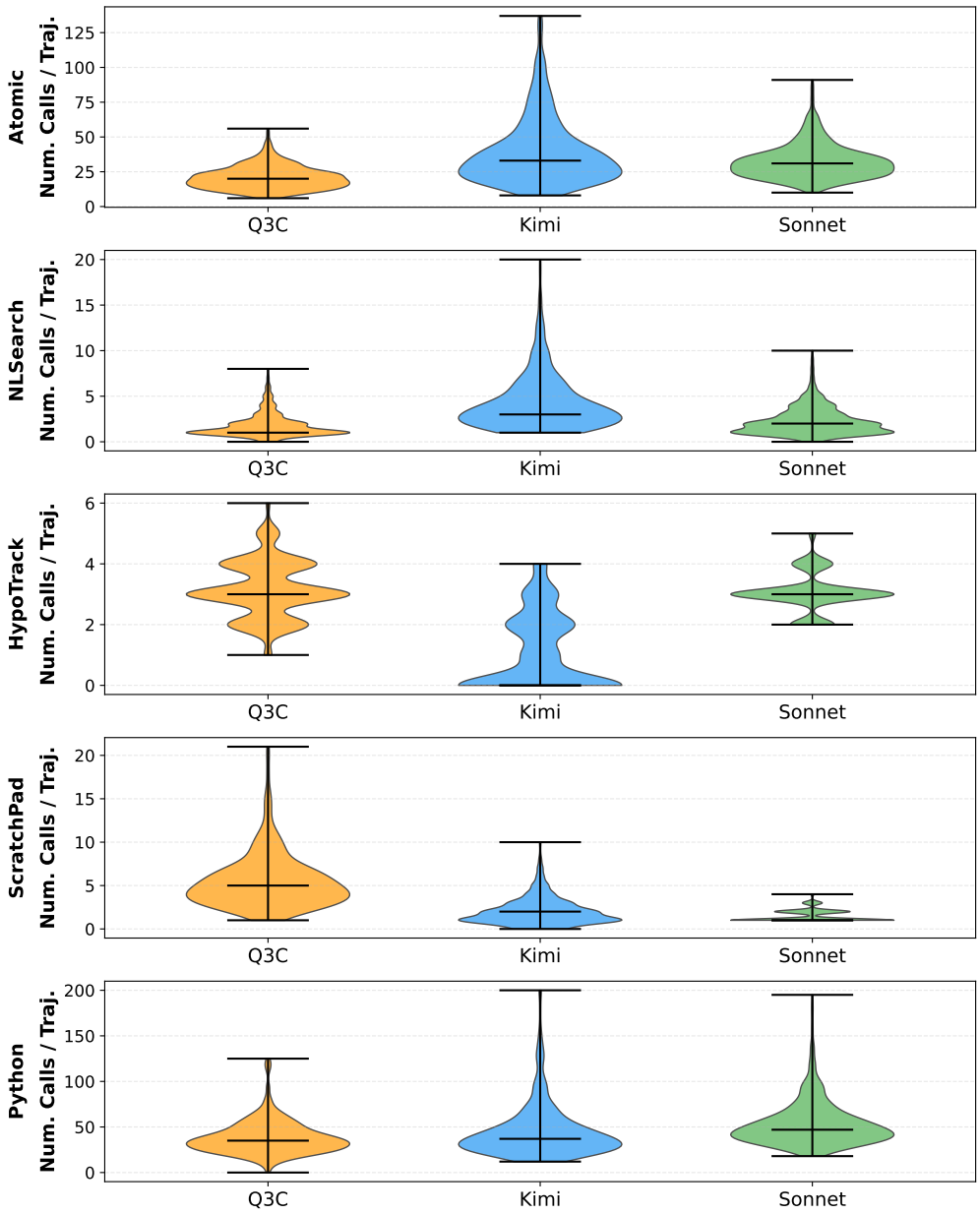



Figure 11: Distribution of tool-call frequency per trajectory under each tool architecture. Each violin shows the number of calls to the architecture-specific tool within a trajectory. Actors use `Atomic` and `Python` frequently across trajectories, whereas `NLSearch`, `HypoTrack`, and `Scratchpad` are invoked much less often.

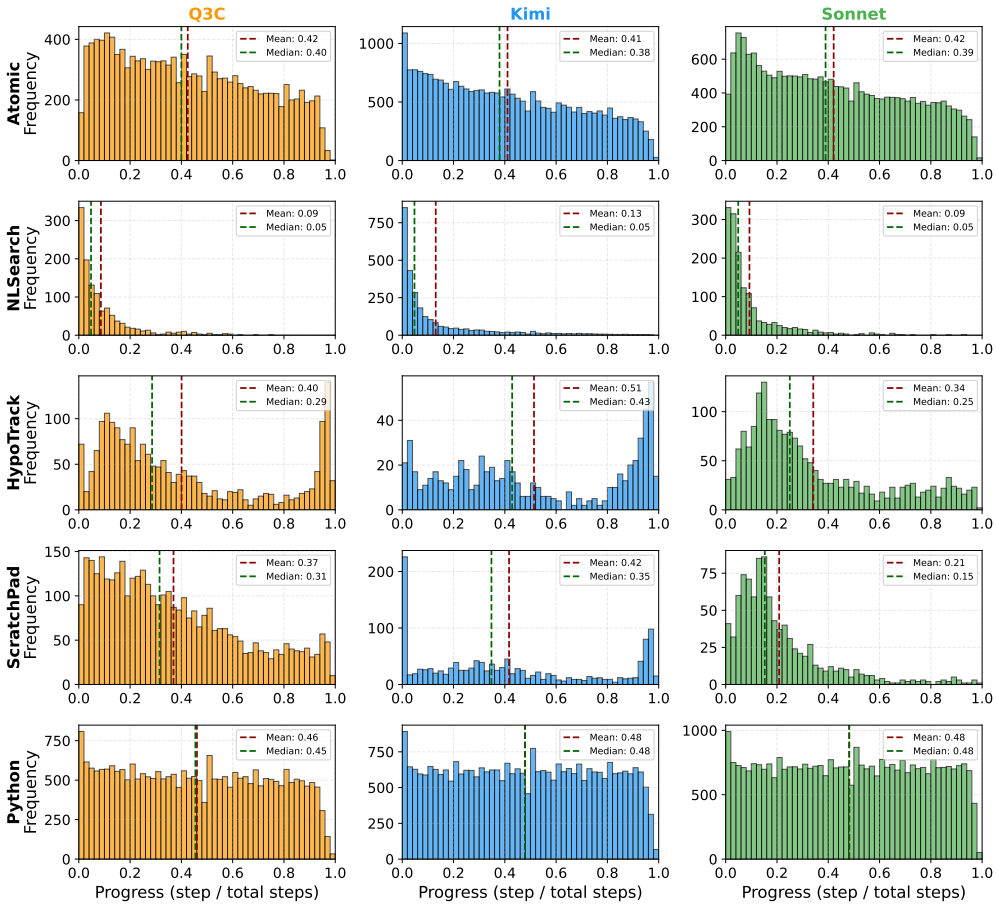



Figure 12: Distribution of tool-call locations within trajectories. For each tool call, we compute its normalized location as *step/total trajectory length*. `Atomic` and `Python` calls are spread broadly across the trajectory, `NLSearch` calls concentrate near the beginning, and `HypoTrack` calls tend to appear near the beginning and the end, consistent with initial hypothesis recording followed by a final status update.